# IT STUDY HUB

## COMPLETE C++ PROGRAMMING SERIES

MODULE 5: OBJECT-ORIENTED PROGRAMMING

**Authors:** Mithila Rani Saha & Dithsa Dutta

**Website:** www.itstudyhub.dpdns.org

# Module Overview

## 1. About This Module

Welcome to **Module 5: Object-Oriented Programming (OOP)** in the IT Study Hub Complete C++ Programming Series. Building upon the procedural constructs, arrays, and memory pointers mastered in Modules 1–4, this module elevates your engineering capability into the industry-standard Object-Oriented paradigm. Authored by Mithila Rani Saha and Dithsa Dutta,

this handbook covers classes, objects, encapsulation, constructors/destructors, inheritance, polymorphism, virtual functions, abstract classes, static members, and friend functions.

## 2. Learning Outcomes

By completing this module, you will be able to:

- Design custom data types using classes, access specifiers (`public`, `private`, `protected`), and encapsulate state.
- Manage object initialization and cleanup using default, parameterized, and copy constructors, alongside destructors and the `this` pointer.
- Implement code reuse and hierarchical organization via single, multilevel, multiple, hierarchical, and hybrid inheritance.
- Achieve compile-time and runtime polymorphism using virtual functions, pure virtual functions, and abstract interfaces.
- Leverage advanced OOP features including `static` class members and `friend` functions.

## 3. Module Coverage

This module comprehensively covers:

- **Core OOP Fundamentals:** Classes, objects, data members, member functions, and encapsulation.
- **Object Lifecycle:** Constructors (default, parameterized, copy), destructors, and the `this` pointer.
- **Inheritance Taxonomy:** Single, multilevel, multiple, hierarchical, and hybrid inheritance patterns.
- **Polymorphism & Abstraction:** Overriding, virtual functions, pure virtual functions, abstract classes, and interfaces.
- **Advanced Constructs:** `static` data/member functions and `friend` functions.

- **Hands-On Project:** Building a complete Object-Oriented Bank Account Management System.
- **Rigorous Assessments:** 15+ practice questions, 20+ viva questions with answers, 15+ OOP interview questions, and coding challenges.

IT STUDY HUB

# 4. Main Notes: Theoretical Foundations

Object-Oriented Programming models software around real-world entities possessing attributes (data) and behaviors (functions). Below are the exhaustive foundational notes for Module 5.

## Concept: Introduction to OOP, Classes, and Objects

**Simple Explanation:** Object-Oriented Programming (OOP) is a programming paradigm based on the concept of 'objects', which encapsulate data and behavior. A class serves as a blueprint or user-defined data type, while an object is an individual runtime instance of that class.

> ***Real-Life Analogy:*** *A class is like an architectural blueprint for a house. You cannot live inside a paper blueprint. An object is the physical house built from that blueprint—you can build multiple houses (objects) from the exact same blueprint.*

```
Class Diagram / Architecture:
Class Blueprint (Car) → Object 1 (Red Sedan), Object 2 (Blue SUV)
```

```
Syntax:
class ClassName {
public:
// Data members and member functions
};
ClassName objectName;
```

```
#include
using namespace std;

class Car {
    public:
        string brand;
```

```
        int speed;

        void displayInfo() {
            cout << "Brand: " << brand << ", Speed: " << speed << "
mph" << endl;
        }
};

int main() {
    Car myCar; // Creating object
    myCar.brand = "Tesla";
    myCar.speed = 150;
    myCar.displayInfo();
    return 0;
}
```

**Output:**
Brand: Tesla, Speed: 150 mph

**Line-by-Line Breakdown:**

- **class Car**: Defines the blueprint for car objects.
- **public:**: Access specifier allowing external access to members.
- **Car myCar;**: Instantiates a runtime object named `myCar`.
- **myCar.displayInfo();**: Invokes the member function on the object instance.

**Real-World Application:** Video game development where every enemy, weapon, and player is an object instantiated from respective classes.

**Common Mistakes:** Forgetting the trailing semicolon `;` at the end of a class definition, which triggers cascading compilation errors.

**Best Practices:** Always capitalize class names (e.g., `BankAccount`, `StudentRecord`) following standard C++ naming conventions.

**Actionable Practice:** Create a `Student` class with name and roll number data members and a print function.

## Concept: Encapsulation, Data Members, Member Functions, and Access Specifiers

**Simple Explanation:** Encapsulation is the bundling of data and methods operating on that data within a single unit (class), restricting direct access using access specifiers (`public`, `private`, `protected`). Private members hide internal state, while public getters and setters provide controlled access.

> ***Real-Life Analogy:*** *Encapsulation is like a capsule medicinal pill or a bank ATM. You interact with simple buttons and a slot (public interface), while complex internal circuitry and cash vaults (private data) are safely hidden away from tampering.*

```
Class Diagram / Architecture:
Private Data → Protected by Class Walls → Public Getter/Setter Interface
```

```
Syntax:
class Account {
private:
double balance;
public:
void setBalance(double b) { balance = b; }
double getBalance() { return balance; }
};
```

```
#include
using namespace std;

class Person {
    private:
        int age;
    public:
        void setAge(int a) {
            if(a > 0) age = a;
```

```
            else age = 0;
        }
        int getAge() {
            return age;
        }
};

int main() {
    Person p;
    p.setAge(22);
    cout << "Valid Age: " << p.getAge() << endl;
    return 0;
}
```

**Output:**
Valid Age: 22

**Line-by-Line Breakdown:**

- **private:int age;**: Hides age data from direct external modification.
- **setAge()**: Validates input before modifying private state.
- **getAge()**: Safely returns private data.

**Real-World Application:** User profile management systems where passwords and sensitive personal IDs are kept strictly private.

**Common Mistakes:** Making all class data members `public`, which violates encapsulation and allows unauthorized external state modification.

**Best Practices:** Keep all data members `private` or `protected`, providing public getter and setter member functions for controlled access.

**Actionable Practice:** Write a `Rectangle` class with private length and width, and public methods for area calculation.

## Concept: Constructors (Default, Parameterized, Copy) and Destructors

**Simple Explanation:** A constructor is a special member function automatically invoked when an object is created, used for initialization. Types include default, parameterized, and copy constructors. A destructor is invoked automatically when an object goes out of scope to clean up resources.

> ***Real-Life Analogy:*** *A constructor is like checking into a hotel room where housekeeping prepares the room with fresh towels and linens upon arrival. A destructor is like checking out, where housekeeping cleans up and locks the room when you leave.*

**Class Diagram / Architecture:**
**Object Creation → Constructor Runs → Lifetime Usage → Destructor Runs → Memory Cleanup**

**Syntax:**

```
class MyClass {
public:
MyClass(); // Default
MyClass(int x); // Parameterized
MyClass(const MyClass &obj); // Copy
~MyClass(); // Destructor
};
```

```
#include
using namespace std;

class Box {
    private:
        int length;
    public:
        Box() { length = 0; cout << "Default ctor\n"; }
        Box(int l) { length = l; cout << "Param ctor\n"; }
```

```
        Box(const Box &b) { length = b.length; cout << "Copy
ctor\n"; }
        ~Box() { cout << "Destructor called for length " << length
<< endl; }
        int getLength() { return length; }
};

int main() {
    Box b1;
    Box b2(10);
    Box b3 = b2;
    return 0;
}
```

```
Output:
Default ctor
Param ctor
Copy ctor
Destructor called for length 10
Destructor called for length 10
Destructor called for length 0
```

**Line-by-Line Breakdown:**

- **Box b1;**: Invokes default constructor.
- **Box b2(10);**: Invokes parameterized constructor.
- **Box b3 = b2;**: Invokes copy constructor.
- Destructors execute automatically in reverse order of creation when main exits.

**Real-World Application:** Opening a database connection inside a constructor and closing it safely inside the destructor (RAII pattern).

**Common Mistakes:** Forgetting to write a deep copy constructor when a class manages dynamic heap memory pointers, causing double-free crashes.

**Best Practices:** Implement a custom copy constructor and destructor whenever your class manages raw dynamic memory pointers.

> **Actionable Practice:** Create a `Book` class with title and price constructors, including a copy constructor.

## Concept: The 'this' Pointer

**Simple Explanation:** The `this` pointer is an implicit pointer available inside non-static member functions that points to the exact object for which the function was invoked. It resolves naming conflicts between class data members and local parameters.

> ***Real-Life Analogy:*** *The `this` pointer is like saying 'myself' or 'this exact person speaking right now' when pointing to yourself in a crowded room.*

**Class Diagram / Architecture:**
**Member function call `obj.setX(10)` → `this` points to `obj` in memory**

**Syntax:**
**void setX(int x) { this->x = x; }**

```
#include
using namespace std;

class Point {
    private:
        int x, y;
    public:
        Point(int x, int y) {
            this->x = x;
            this->y = y;
        }
        void print() {
            cout << "Coordinate: (" << this->x << ", " << this->y <<
```

```
")" << endl;
        }
};

int main() {
    Point pt(5, 12);
    pt.print();
    return 0;
}
```

**Output:**
```
Coordinate: (5, 12)
```

**Line-by-Line Breakdown:**

- **this->x = x;**: Distinguishes between the class data member `x` and the parameter `x`.

**Real-World Application:** Method chaining in fluent API design (returning `*this` from member functions).

**Common Mistakes:** Attempting to use the `this` pointer inside static member functions (static functions do not belong to a specific object instance).

**Best Practices:** Use `this->` explicitly when local parameter names shadow class data member names to maintain code clarity.

**Actionable Practice:** Write a class with method chaining returning `*this`.

## Concept: Inheritance (Single, Multilevel, Multiple, Hierarchical, Hybrid)

**Simple Explanation:** Inheritance is an OOP mechanism allowing a new class (derived/child class) to inherit properties and behaviors of an existing class (base/parent class). C++ supports single, multilevel, multiple, hierarchical, and hybrid inheritance.

> ***Real-Life Analogy:*** *Inheritance is like biological genetics: a child inherits eye color, height, and traits from their parents, and can also develop unique new traits of their own.*

**Class Diagram / Architecture:**
**Base Class (Vehicle) → Derived Class (Car) inherits properties**

**Syntax:**
**class Derived : access_specifier Base { };**

```
#include
using namespace std;

class Animal {
    public:
        void eat() { cout << "Eating...\n"; }
};

class Dog : public Animal {
    public:
        void bark() { cout << "Barking...\n"; }
};

int main() {
    Dog myDog;
    myDog.eat();
    myDog.bark();
    return 0;
}
```

```
Output:
Eating...
Barking...
```

**Line-by-Line Breakdown:**

- **class Dog : public Animal**: `Dog` inherits all public members of `Animal`.
- `myDog` successfully accesses both inherited `eat()` and native `bark()` methods.

**Real-World Application:** Designing UI component libraries where buttons, text boxes, and sliders all inherit from a base `UIComponent` class.

**Common Mistakes:** Using `private` inheritance when public 'is-a' inheritance relationships are intended.

**Best Practices:** Use public inheritance to model true 'is-a' relationships (e.g., a Dog *is an* Animal).

**Actionable Practice:** Create a multilevel inheritance hierarchy: `Person` -> `Employee` -> `Manager`.

## Concept: Polymorphism, Function Overriding, and Virtual Functions

**Simple Explanation:** Polymorphism means 'many forms'. Compile-time polymorphism is achieved via function/operator overloading, while runtime polymorphism is achieved via inheritance, virtual functions, and pointers/references, allowing base class pointers to execute derived class methods dynamically.

> ***Real-Life Analogy:*** *Polymorphism is like a button labeled 'Press'. Depending on whether you press it on a coffee machine, an elevator, or a computer keyboard, the exact same action ('Press') triggers completely different behaviors.*

**Class Diagram / Architecture:**
**Base Pointer `Animal* p` → Points to `Dog` object → Executes `Dog::sound()` dynamically**

**Syntax:**

```
class Base {
public:
virtual void show() { cout << "Base\n"; }
};
class Derived : public Base {
public:
void show() override { cout << "Derived\n"; }
};
```

```
#include
using namespace std;

class Shape {
    public:
        virtual void draw() {
            cout << "Drawing generic shape.\n";
        }
};

class Circle : public Shape {
    public:
        void draw() override {
            cout << "Drawing Circle.\n";
        }
};

int main() {
    Shape *s;
    Circle c;
```

```
    s = &c;
    s->draw(); // Runtime polymorphism via virtual function
    return 0;
}
```

> **Output:**
> `Drawing Circle.`

**Line-by-Line Breakdown:**

- **virtual void draw()**: Enables dynamic dispatch (vtable lookup).
- **s->draw()**: Executes `Circle::draw()` at runtime even though `s` is a `Shape*` pointer.

**Real-World Application:** Graphics engines rendering mixed shapes (`Circle`, `Rectangle`, `Triangle`) by iterating through an array of base `Shape*` pointers.

> **Common Mistakes:** Forgetting to declare base class destructors as `virtual` when using polymorphic deletion, leading to resource leaks.

> **Best Practices:** Always use the `override` keyword when overriding virtual functions to let the compiler catch signature mismatch typos.

> **Actionable Practice:** Create an `Employee` base class with a virtual `calculateSalary()` function overridden in derived classes.

## Concept: Pure Virtual Functions, Abstract Classes, and Interfaces

**Simple Explanation:** A pure virtual function is a virtual function initialized to zero (`= 0`). A class containing at least one pure virtual function is an abstract class and cannot be instantiated directly. Abstract classes act as interfaces defining mandatory contracts for derived classes.

> ***Real-Life Analogy:*** *An abstract class is like a corporate job description for 'Remote Worker'. You cannot hire a generic 'Remote Worker' directly, but every specific employee hired must fulfill all duties outlined in that description.*

**Class Diagram / Architecture:**
**Abstract Class (Interface) → Pure Virtual `= 0` → Must be implemented by Derived classes**

**Syntax:**

```
class Interface {
public:
virtual void run() = 0; // Pure virtual
};
```

```
#include
using namespace std;

class PaymentGateway {
    public:
        virtual void processPayment(double amount) = 0; // Pure
virtual
};

class PayPal : public PaymentGateway {
    public:
        void processPayment(double amount) override {
            cout << "Processing $" << amount << " via PayPal.\n";
        }
```

```
};

int main() {
    // PaymentGateway pg; // Error: cannot instantiate abstract
class
    PaymentGateway *gateway = new PayPal();
    gateway->processPayment(150.75);
    delete gateway;
    return 0;
}
```

> **Output:**
> `Processing $150.75 via PayPal.`

**Line-by-Line Breakdown:**

- **virtual void processPayment(double) = 0;**: Makes `PaymentGateway` an abstract class.
- **PayPal** overrides the pure virtual function, allowing instantiation.

**Real-World Application:** Plugin architectures where an application defines a pure abstract plugin interface that all third-party plugins must implement.

> **Common Mistakes:** Failing to implement all pure virtual functions in a derived class, which leaves the derived class abstract and uninstantiable.

> **Best Practices:** Use abstract classes to enforce strict architectural design contracts across large software teams.

> **Actionable Practice:** Create an abstract `RemoteControl` class with pure virtual `pressPowerButton()`.

## Concept: Friend Functions and Static Members

**Simple Explanation:** A `friend` function is a non-member function granted special access to private and protected members of a class. `static` members (variables or functions) belong to the class as a whole rather than to any specific object instance, sharing a single copy across all objects.

> ***Real-Life Analogy:*** *A `friend` function is like a trusted family lawyer who is granted access to private family vault documents. A `static` member is like a community bulletin board shared by all residents in an apartment building.*

**Class Diagram / Architecture:**
**Friend Function → Accesses private class vaults | Static Variable → Shared across all object instances**

**Syntax:**

```
class Box {
friend void showBox(Box b);
static int objectCount;
};
```

```
#include
using namespace std;

class Counter {
    private:
        static int count;
    public:
        Counter() { count++; }
        static int getCount() { return count; }
};

int Counter::count = 0; // Initialize static member

int main() {
    Counter c1, c2, c3;
    cout << "Total objects created: " << Counter::getCount() <<
```

```
endl;
    return 0;
}
```

**Output:**
```
Total objects created: 3
```

**Line-by-Line Breakdown:**

- **static int count;**: Declared inside class, initialized outside.
- **Counter::getCount()**: Accesses static member without requiring an object instance.

**Real-World Application:** Tracking active network connections or total active database sessions across an application.

**Common Mistakes:** Forgetting to allocate and initialize static data members outside the class definition in the global scope.

**Best Practices:** Use static member functions only when behavior does not depend on instance-specific object data.

**Actionable Practice:** Create a class with a static variable counting how many instances currently exist.

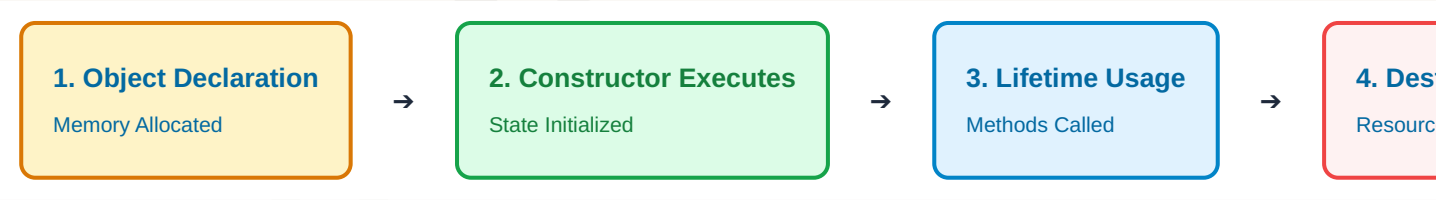
# Visual Workflows & Architecture

Architectural diagrams mapping class-object relationships, constructor lifecycles, inheritance hierarchies, and virtual function dynamic dispatch tables.

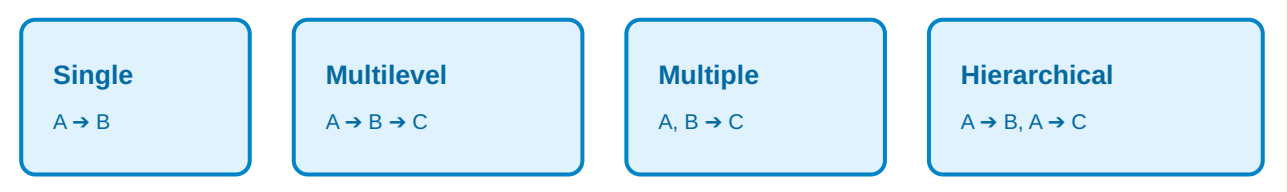
### Diagram 1: Class vs Object Memory Concept



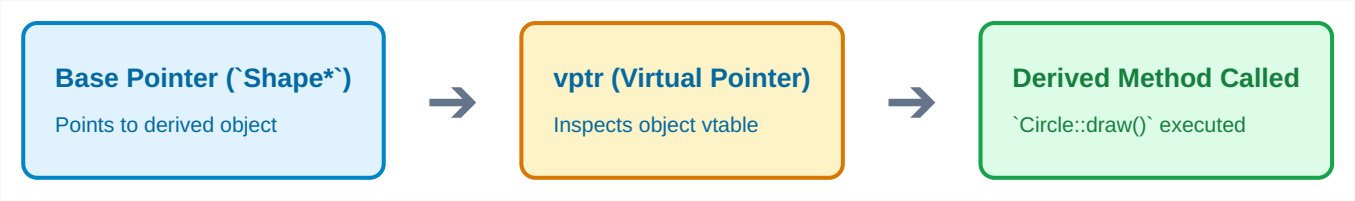

### Diagram 2: Constructor & Destructor Object Lifecycle




### Diagram 3: Inheritance Taxonomy (Types)



## Diagram 4: Virtual Function Mechanism (Dynamic Dispatch)




IT STUDY HUB

# Comparative Analysis

Professional C++ software engineering requires mastering the distinctions between foundational OOP constructs.

### Table 1: Class vs Object

| Feature | Class | Object |
|---|---|---|
| **Definition** | A blueprint or user-defined data type. | An instance of a class occupying physical RAM. |
| **Allocation** | Does not consume memory when declared. | Allocates memory for data members upon instantiation. |
| **Quantity** | Declared once in code. | Can be instantiated many times. |

### Table 2: Constructor vs Destructor

| Feature | Constructor | Destructor |
|---|---|---|
| **Purpose** | Initializes object state and allocates resources. | Cleans up resources and deallocates memory. |
| **Naming** | Same name as the class. | Same name prefixed with a tilde (`~`). |
| **Parameters** | Can accept parameters (overloaded). | Takes zero parameters. |

### Table 3: Default vs Parameterized vs Copy Constructor

| Constructor Type | Parameters | Primary Use Case |
|---|---|---|
| **Default** | None | Initializes objects with default/zero values. |
| **Parameterized** | Custom parameters | Initializes objects with specific custom values. |
| **Copy** | Const reference to same class (`const T&`) | Creates a new object as a copy of an existing object. |

### Table 4: public vs private vs protected

| Access Specifier | Within Class | Derived Class | Outside Class |
|---|---|---|---|
| **public** | Yes | Yes | Yes |
| **protected** | Yes | Yes | No |
| **private** | Yes | No | No |

### Table 5: Overloading vs Overriding

| Feature | Function Overloading | Function Overriding |
|---|---|---|
| **Scope** | Same class scope. | Base and derived class inheritance scope. |
| **Signatures** | Must have different parameter signatures. | Must have identical parameter signatures. |
| **Binding** | Compile-time static binding. | Runtime dynamic binding (via `virtual`). |

### Table 6: Compile-time vs Runtime Polymorphism

| Aspect | Compile-Time (Static) | Runtime (Dynamic) |
|---|---|---|
| **Mechanism** | Function/operator overloading, templates. | Virtual functions, inheritance. |
| **Speed** | Faster execution (resolved at compile time). | Slight vtable lookup overhead at runtime. |

### Table 7: Virtual vs Pure Virtual Function

| Feature | Virtual Function | Pure Virtual Function |
|---|---|---|
| **Implementation** | Has a default body implementation in base class. | No body; initialized to zero (`= 0`). |
| **Instantiation** | Base class can be instantiated. | Makes base class abstract (cannot be instantiated). |

### Table 8: Class Member vs Static Member

| Aspect | Instance Member | Static Member |
|---|---|---|
| **Storage** | Separate copy for every object instance. | Single shared copy for the entire class. |
| **Access** | Accessed via object instance (`obj.member`). | Accessed via class scope (`Class::member`). |

# HANDS-ON PROJECT: BANK ACCOUNT MANAGEMENT SYSTEM

In this comprehensive hands-on project, we combine classes, encapsulation, constructors, inheritance, method overriding, runtime polymorphism, static account tracking, and exception-safe design into a professional banking utility suite.

## Project Specification

**Problem Statement:** Build a complete Object-Oriented Bank Account Management System featuring a base `BankAccount` class, derived account types (`SavingsAccount`, `CheckingAccount`), encapsulation, runtime polymorphism for interest/fee calculations, static account tracking, and safe deposit/withdrawal transaction handling.

```
#include <iostream>
#include <string>
using namespace std;

class BankAccount {
    protected:
        string accountNumber;
        string customerName;
        double balance;
        static int totalAccounts;

    public:
        BankAccount(string accNum, string name, double
initialBalance) {
            accountNumber = accNum;
            customerName = name;
            balance = (initialBalance >= 0) ? initialBalance : 0;
            totalAccounts++;
        }

        virtual ~BankAccount() {
            totalAccounts--;
```

```
        }

        virtual void deposit(double amount) {
            if(amount > 0) {
                balance += amount;
                cout << "Deposited $" << amount << " successfully.
\n";
            } else {
                cout << "Invalid deposit amount.\n";
            }
        }

        virtual void withdraw(double amount) {
            if(amount > 0 && amount <= balance) {
                balance -= amount;
                cout << "Withdrew $" << amount << " successfully.
\n";
            } else {
                cout << "Withdrawal failed: Insufficient funds or
invalid amount.\n";
            }
        }

        double getBalance() const { return balance; }
        string getAccountNum() const { return accountNumber; }
        string getName() const { return customerName; }

        static int getTotalAccounts() { return totalAccounts; }

        virtual void displayDetails() const {
            cout << "Account: " << accountNumber << " | Holder: " <<
customerName << " | Balance: $" << balance << endl;
        }
};

int BankAccount::totalAccounts = 0;

class SavingsAccount : public BankAccount {
    private:
        double interestRate;

    public:
        SavingsAccount(string accNum, string name, double
initialBalance, double rate)
            : BankAccount(accNum, name, initialBalance),
interestRate(rate) {}
```

```
        void addInterest() {
            double interest = balance * (interestRate / 100.0);
            balance += interest;
            cout << "Interest added: $" << interest << ". New
Balance: $" << balance << endl;
        }

        void displayDetails() const override {
            cout << "[Savings] ";
            BankAccount::displayDetails();
        }
};

class CheckingAccount : public BankAccount {
    private:
        double transactionFee;

    public:
        CheckingAccount(string accNum, string name, double
initialBalance, double fee)
            : BankAccount(accNum, name, initialBalance),
transactionFee(fee) {}

        void withdraw(double amount) override {
            double totalReq = amount + transactionFee;
            if(totalReq <= balance) {
                balance -= totalReq;
                cout << "Withdrew $" << amount << " with $" <<
transactionFee << " fee. Balance: $" << balance << endl;
            } else {
                cout << "Checking withdrawal failed due to fee/
balance constraints.\n";
            }
        }

        void displayDetails() const override {
            cout << "[Checking] ";
            BankAccount::displayDetails();
        }
};

int main() {
    cout << "======================================" << endl;
    cout << " IT STUDY HUB BANK MANAGEMENT SYSTEM    " << endl;
    cout << "======================================" << endl;
```

```
    BankAccount *acc1 = new SavingsAccount("SAV-101", "Mithila
Rani", 5000.0, 4.5);
    BankAccount *acc2 = new CheckingAccount("CHK-202", "Dithsa
Dutta", 2500.0, 2.0);

    acc1->displayDetails();
    acc2->displayDetails();

    cout << "\nTotal Active Accounts: " <<
BankAccount::getTotalAccounts() << endl;

    cout << "\n--- Executing Transactions ---" << endl;
    acc1->deposit(1000.0);
    acc2->withdraw(500.0);

    // Dynamic cast for savings specific method
    SavingsAccount *sav = dynamic_cast<SavingsAccount*>(acc1);
    if(sav) {
        sav->addInterest();
    }

    cout << "\n--- Final Account Summary ---" << endl;
    acc1->displayDetails();
    acc2->displayDetails();

    delete acc1;
    delete acc2;

    cout << "Total Active Accounts after cleanup: " <<
BankAccount::getTotalAccounts() << endl;
    return 0;
}
```

**Expected Output:**

```
========================================
 IT STUDY HUB BANK MANAGEMENT SYSTEM
========================================
[Savings] Account: SAV-101 | Holder: Mithila Rani | Balance:
$5000
[Checking] Account: CHK-202 | Holder: Dithsa Dutta | Balance:
$2500
```

```
Total Active Accounts: 2

--- Executing Transactions ---
Deposited $1000 successfully.
Withdrew $500 with $2 fee. Balance: $1998
Interest added: $270. New Balance: $6270

--- Final Account Summary ---
[Savings] Account: SAV-101 | Holder: Mithila Rani | Balance:
$6270
[Checking] Account: CHK-202 | Holder: Dithsa Dutta | Balance:
$1998
Total Active Accounts after cleanup: 0
```

**Step-by-Step Explanation:**

- `BankAccount` serves as the encapsulated base class managing core balance state, protected variables, and static account tracking.
- `SavingsAccount` and `CheckingAccount` inherit from `BankAccount`, demonstrating polymorphism via method overriding (`displayDetails()`, `withdraw()`).
- Virtual destructors ensure proper cleanup when base pointers (`BankAccount*`) delete derived objects.
- Static member `totalAccounts` accurately tracks active object count across the application lifecycle.

**Practice Challenges:**

- Add a `FixedDeposit` account type with withdrawal locking constraints.
- Implement exception handling for negative deposits and overdraft attempts.

# Module Assessments

Test your mastery of Object-Oriented Programming with these rigorously curated questions.

## 15+ Practice Questions

1. What is the difference between a class and an object?
2. Explain encapsulation and how access specifiers enforce it.
3. What is the role of a constructor, and when is it invoked?
4. Differentiate between default, parameterized, and copy constructors.
5. What is the purpose of a destructor? When must it be declared virtual?
6. Explain the `this` pointer and where it can be used.
7. What are the different types of inheritance supported in C++?
8. What is function overriding, and how does it differ from function overloading?
9. Explain runtime polymorphism and the role of virtual functions.
10. What is a pure virtual function, and what makes a class abstract?
11. Can an abstract class be instantiated? Why or why not?
12. What is a `friend` function, and what are its security implications?
13. Explain `static` data members and `static` member functions.
14. Why can't a static member function access the `this` pointer?
15. Write a class hierarchy demonstrating multilevel inheritance.

## 20+ Viva Questions (With Answers)

**Q1: What is Object-Oriented Programming (OOP)?**

> A programming paradigm based on classes and objects that bundles data and behaviors together.

**Q2: What is a class?**

> A user-defined blueprint or prototype from which objects are created.

**Q3: What is an object?**

> An instance of a class that occupies physical RAM storage.

**Q4: What is encapsulation?**

> The wrapping up of data and methods into a single unit, hiding internal state via access specifiers.

**Q5: What are the three access specifiers in C++?**

> `public`, `private`, and `protected`.

**Q6: What is a constructor?**

> A special member function automatically called when an object is instantiated to initialize its state.

**Q7: Can constructors be overloaded?**

> Yes, a class can have multiple constructors with differing parameter signatures.

**Q8: What is a copy constructor?**

> A constructor that initializes an object using another existing object of the same class (`const T&`).

**Q9: What is a destructor?**

> A member function prefixed with `~` that executes automatically when an object is destroyed.

**Q10: What is the `this` pointer?**

> An implicit pointer available in non-static member functions pointing to the current object instance.

**Q11: What is inheritance?**

> A mechanism where a derived class acquires properties and behaviors of a base class.

**Q12: What is multiple inheritance?**

> When a derived class inherits from more than one base class.

**Q13: What is polymorphism?**

> The ability of functions or objects to take on multiple forms.

**Q14: What is a virtual function?**

> A base class member function declared with `virtual` that enables runtime dynamic dispatch.

**Q15: What is a pure virtual function?**

> A virtual function set to zero (`= 0`) that has no implementation in the base class.

**Q16: What is an abstract class?**

> A class containing at least one pure virtual function, making it uninstantiable.

**Q17: What is a friend function?**

> A non-member function granted access to private and protected members of a class.

**Q18: What is a static member?**

> A data member or function shared across all instances of a class rather than belonging to any single object.

**Q19: Can static functions be virtual?**

> No, virtual functions rely on object instance vptrs, whereas static functions belong to the class globally.

**Q20: Why must base class destructors be virtual in polymorphic hierarchies?**

> To ensure derived class destructors execute correctly when deleting objects via base class pointers, preventing memory leaks.

## 15+ Technical OOP Interview Questions

1. **Explain the Four Pillars of Object-Oriented Programming.** Detail Encapsulation, Abstraction, Inheritance, and Polymorphism with real-world examples.
2. **What is the Diamond Problem in multiple inheritance, and how does C++ solve it using virtual inheritance?** Explain ambiguity resolution and shared base subobjects.
3. **How do virtual functions work under the hood? Explain vptr and vtable.** Describe how the compiler generates virtual method tables for dynamic dispatch.

4. **What is the Rule of Three / Rule of Five in C++ class design?** Discuss destructor, copy constructor, copy assignment operator, move constructor, and move assignment operator.
5. **Can a constructor or destructor be virtual? Why or why not?** Explain why constructors cannot be virtual (object type not yet established) and why base destructors should be virtual.
6. **What is object slicing in C++ inheritance? Explain when and why it happens.** Detail how assigning a derived object directly to a base object truncates derived member data.
7. **When should you use private inheritance vs composition (has-a relationship)?** Discuss implementation inheritance ("implemented-in-terms-of") versus containment.
8. **What is the difference between an abstract class and an interface in C++?** Explain that C++ uses pure abstract classes to model interfaces as it lacks a native `interface` keyword.
9. **Explain copy elision and Return Value Optimization (RVO) in modern C++.** Discuss how compilers optimize away redundant constructor/destructor overhead.
10. **What is the purpose of `explicit` constructors?** Explain how `explicit` prevents unintended implicit type conversions during object initialization.
11. **How do you implement a singleton design pattern in C++?** Discuss private constructors, deleted copy constructors, and static instance methods.
12. **What is dynamic_cast, and when should you use it?** Explain safe downcasting in polymorphic class hierarchies using runtime type information (RTTI).
13. **Why are friend functions considered a potential violation of encapsulation?** Discuss trade-offs between tight coupling and performance/access utility.
14. **What is shallow copy vs deep copy, and when does the default compiler-generated copy constructor fail?** Explain dynamic heap pointer duplication issues.
15. **Explain name mangling in C++ member functions and how `extern "C"` interacts with it.** Detail how compiler function signatures encode parameter types.

# Module 5 Wrap-Up

## Key Terms Glossary

- **Class:** Blueprint defining data and methods for objects.
- **Object:** Runtime instance of a class.
- **Encapsulation:** Bundling state and restricting direct access.
- **Constructor:** Special member function initializing object state.
- **Destructor:** Cleanup member function freeing resources upon destruction.
- **Polymorphism:** Ability of code to exhibit multiple behaviors dynamically.
- **Virtual Function:** Base function enabling runtime dynamic method dispatch.
- **Abstract Class:** Class with pure virtual functions acting as an interface.

## Quick Revision Summary

Module 5 introduces Object-Oriented Programming. Classes serve as blueprints for runtime objects. Encapsulation hides state behind access specifiers (`public`, `private`, `protected`). Object lifecycles are managed by constructors and destructors. Inheritance enables code reuse across single, multilevel, multiple, and hierarchical structures. Polymorphism, achieved via virtual functions and pure virtual functions, enables dynamic dispatch and abstract interfaces. Advanced features include `static` members and `friend` functions.

## OOP Coding Challenges

Complete these challenges before moving to Module 6:

- **Challenge 1: Shape Hierarchy.T** Create an abstract `Shape` class with pure virtual `calculateArea()` and `calculatePerimeter()` methods, implemented by `Circle`, `Rectangle`, and `Triangle` derived classes.
- **Challenge 2: Smart Vehicle Registry.T** Design a vehicle inheritance hierarchy with static instance counting and virtual method overriding for speed boost calculations.
- **Challenge 3: Deep Copy Manager.T** Write a custom class managing a dynamic integer array pointer on the heap, implementing a robust deep copy constructor and destructor without memory leaks.

## CONGRATULATIONS ON COMPLETING MODULE 5!

WHAT COMES NEXT:

**MODULE 6 — EXCEPTION HANDLING, FILE HANDLING & STL BASICS**